

Big Data

Cloud Provider Analytics

Consigna consolidada del proyecto y de las tres instancias de evaluación

DOCENTE Prof. (Ad.) Diego Mosquera

MODALIDAD 100 % remota

PROYECTO ETL + Streaming + Serving en Cassandra

EVALUACIÓN Dos entregas parciales acumulativas y un MVP final

PRIMERA ENTREGA 28/09/2026 · 18:30 h

SEGUNDA ENTREGA 16/11/2026 · 18:30 h

ENTREGA FINAL 07/12/2026 · 21:30 h

A

Ficha del documento

Identificación, alcance y criterios de uso

Campo	Definición
Asignatura	Big Data · curso electivo
Institución	Instituto Tecnológico de Buenos Aires (ITBA)
Docente responsable	Diego Mosquera · Profesor Adjunto
Período	Segundo cuatrimestre de 2026
Modalidad	100 % remota, incluidas las evaluaciones y la instancia final
Proyecto común	Cloud Provider Analytics
Tecnologías centrales	PySpark, Structured Streaming, Parquet y Cassandra/AstraDB
Estrategia evaluativa	Proyecto integrador único: dos entregas parciales acumulativas y un MVP final

Propósito del documento

Este documento consolida en una única consigna el caso de uso Cloud Provider Analytics, los requisitos técnicos, los artefactos de entrega, los criterios de aceptación y las rúbricas de las tres instancias de evaluación..

Contenido

Sección	Contenido
1	Lógica del proyecto y calendario de evaluación
2	Caso de uso: Cloud Provider Analytics
3	Datos provistos y restricciones
4	Arquitectura y requisitos técnicos comunes
5	Primera evaluación parcial: diseño y fundación
6	Segunda evaluación parcial: implementación técnica
7	Evaluación final: MVP integrado
8	Estándares de implementación y entrega
9	Anexos: checklists de entrega

1

Lógica del proyecto y calendario de evaluación

Una única solución que evoluciona de diseño conceptual a MVP reproducible

1.1 Principio acumulativo

Las evaluaciones no son productos independientes. Cada instancia reutiliza, corrige y amplía la anterior. La primera entrega valida la comprensión del problema y la viabilidad arquitectónica; la segunda exige un pipeline ejecutable y evidencia técnica; la instancia final integra la solución completa, su reproducibilidad y la defensa.

- Primera etapa: comprender el caso, diseñar la arquitectura y fundar el Data Lake.
- Segunda etapa: implementar un end-to-end mínimo con ingesta batch y streaming, Spark, Gold y Cassandra.
- Etapa final: cerrar el flujo end-to-end, completar los dominios analíticos, fortalecer gobierno, calidad, pruebas y documentación, y defender técnicamente la solución.

1.2 Calendario consolidado

Instancia	Foco	Fecha y hora límite	Validación
Primera evaluación parcial	Diseño y fundación de datos	Lunes 28/09/2026 · 18:30 h	Revisión y feedback en la clase del 28/09
Segunda evaluación parcial	Implementación técnica	Lunes 16/11/2026 · 18:30 h	Revisión y feedback en la clase del 16/11
Evaluación final	MVP integrado y defensa	Lunes 07/12/2026 · 21:30 h	Revisión y feedback

Regla de corte

La versión evaluada será la disponible en el repositorio y en los artefactos entregados a la hora límite de cada instancia. Los cambios posteriores deberán quedar identificados como correcciones derivadas del feedback.

1.3 Gestión de evidencias

- Repositorio versionado con estructura y README acordados.
- Diagramas con versión y fecha, consistentes con la implementación real.
- Notebooks, scripts o jobs ejecutables.
- Datos de muestra y mecanismo reproducible para obtenerlos o generarlos.
- Capturas, logs o salidas que demuestren ejecución.
- Registro de decisiones técnicas, supuestos y trade-offs.
- Plan de correcciones después de la primera entrega y backlog priorizado después de la segunda.

2

Caso de uso: Cloud Provider Analytics

ETL, streaming y serving para FinOps, Soporte y Producto

2.1 Contexto

El equipo representa al área de datos de un proveedor de nube. Su misión es ingestar, limpiar, conformar y publicar datos de clientes para analítica de FinOps, Soporte y Producto. La organización requiere dos capacidades complementarias:

1. Near real-time para métricas operativas de uso, consumo y costos incrementales.
2. Procesamiento batch diario o mensual para maestros de CRM, facturación y fuentes de referencia.

Los datos aterrizan crudos con inconsistencias, nulos, tipos ambiguos, anomalías y una evolución de esquema a mitad del histórico. En la versión 2 de los eventos aparecen campos como `carbon_kg` y `genai_tokens`.

2.2 Objetivo técnico

Resultado esperado

Construir un pipeline con PySpark, ejecutable en Google Colab o en un entorno equivalente validado, usando Parquet como almacenamiento intermedio gestionado por Spark y publicando marts analíticos en Cassandra/AstraDB para su consulta desde herramientas de visualización.

2.3 Usuarios y dominios de consumo

Dominio	Necesidad analítica
FinOps	Costos, consumo, revenue, créditos, impuestos, anomalías y eficiencia por organización y servicio.
Soporte	Volumen de tickets, severidad, cumplimiento de SLA y CSAT por organización y fecha.
Producto / Usage	Uso de servicios, requests, métricas operativas, tokens GenAI y carbono cuando estén disponibles.

3

Datos provistos y restricciones

Fuentes de Landing, evolución de esquema y problemas de calidad

3.1 Fuentes disponibles

Fuente	Descripción
customers_orgs.csv	Clientes/tenants: industria, región, plan, NPS; incluye nulos y valores ruidosos.
users.csv	Usuarios por organización: roles, actividad y timestamps.
resources.csv	Recursos cloud por servicio: compute, storage, database, networking, analytics y genai.
support_tickets.csv	Tickets: categoría, severidad, SLA, CSAT y estado abierto/cerrado; incluye nulos.
marketing_touches.csv	Interacciones de marketing: canales y conversiones.
nps_surveys.csv	Encuestas NPS temporales por organización; incluye nulos.
billing_monthly.csv	Facturación de tres meses: impuestos, créditos/ajustes y monedas.
usage_events_stream/*.jsonl	Eventos de uso fragmentados para simular micro-lotes y ser procesados con Structured Streaming.

3.2 Condiciones que deben ser tratadas

- Evolución de esquema: schema_version=2 incorpora carbon_kg y, para el servicio genai, genai_tokens.
- Tipos ambiguos: algunos campos numéricos pueden llegar como texto y deben castearse con fallback controlado.
- Valores nulos y atípicos: existen costos negativos ocasionales y spikes de magnitud elevada.
- Fragmentación intencional de eventos para simular el comportamiento de micro-lotes.
- Datos crudos inmutables: los archivos de Landing no deben modificarse.

4

Arquitectura y requisitos técnicos comunes

Responsabilidades mínimas que atraviesan las tres instancias

4.1 Cadena lógica de referencia

Cadena común

Fuentes -> Ingesta -> Data Lake -> Procesamiento batch/streaming -> Serving -> Analítica/ML -> Consumo, con gobierno, calidad, seguridad, metadatos y observabilidad como capacidades transversales.

4.2 Zonas del Data Lake

Zona	Responsabilidad
Landing · Raw inmutable	Archivos originales. No modificar.
Bronze · Raw estándar	Mismo grano de la fuente, tipificación explícita, deduplicación cuando corresponda y columnas técnicas ingest_ts y source_file.
Silver · Conformado	Normalización, joins, tratamiento de nulos y outliers, compatibilización de versiones de esquema y SCD cuando aplique.
Gold · Marts de negocio	Tablas o vistas orientadas a FinOps, Soporte y Producto/Usage, preparadas para servir en Cassandra/AstraDB.

El formato intermedio será Parquet particionado, por ejemplo por fecha y/o servicio. La estrategia de particionamiento, retención, naming y promoción entre zonas debe quedar justificada.

4.3 Patrón arquitectónico

Opción	Criterio
Lambda	Batch para maestros, facturación y encuestas; streaming para usage_events_stream con ventanas, watermarks e idempotencia.
Kappa	Tratamiento de las fuentes como streams, con re-stream o backfill para derivar las capas Silver y Gold.
Híbrido	Combinación explícita y justificada cuando resuelva mejor las latencias, los costos y la complejidad del caso.

Requisito invariable

Independientemente del patrón elegido, la solución debe implementar al menos streaming de eventos y batch de maestros/facturación, explicando por qué la elección es adecuada.

4.4 Requisitos técnicos obligatorios del proyecto completo

Capacidad	Requisito
Ingesta batch	Leer CSV/JSON desde Landing y escribir Bronze Parquet particionado, con esquemas explícitos y columnas técnicas.
Ingesta streaming	Structured Streaming desde directorio JSONL, esquema explícito, watermark, deduplicación por event_id, late data y checkpointing.

Capacidad	Requisito
Calidad	Reglas verificables, separación o bloqueo de registros inválidos y quarantine en Parquet.
Silver	Normalización de números, fechas, regiones y servicios; joins con dimensiones; tratamiento de nulos/outliers; compatibilidad v1/v2.
Features	daily_cost_usd, requests, cpu_hours, storage_gb_hours, genai_tokens y carbon_kg cuando existan.
Anomalías	Flags o scores usando métodos justificados, por ejemplo z-score, MAD o percentiles.
Gold	Marts orientados a FinOps, Soporte y Producto/Usage con granos claros.
Serving	Keyspace Cassandra/AstraDB, tablas modeladas query-first y carga desde Spark mediante conector o foreachBatch/driver.
Idempotencia	Reprocesamiento sin duplicados mediante checkpoints, claves naturales y upserts.
Performance	Particionado sensato, control de archivos, coalesce/repartition y evidencias de tamaños/rutas.
Gobierno	Calidad, metadatos, linaje, responsabilidades, seguridad y observabilidad como capacidades transversales.
Documentación	Diagrama, diccionario de datos, decisiones, trade-offs, pruebas, Quickstart y evidencias de ejecución.

5

Primera evaluación parcial

Diseño y fundación de datos

FECHA Y HORA LÍMITE

Lunes 28 de septiembre de 2026 · 18:30 h

La jornada de clase del 28/09 se utilizará para revisión, feedback técnico y elaboración del plan de correcciones.

5.1 Objetivo

Validar que el equipo comprende el caso, justifica la necesidad de Big Data y propone una arquitectura viable antes de profundizar la implementación. La entrega debe ser concisa, visual y accionable; no se espera una implementación profunda ni over-engineering en esta etapa.

5.2 Alcance obligatorio

1. Interpretación del problema, usuarios, preguntas principales y objetivos medibles.
2. Justificación de la necesidad de Big Data mediante volumen, velocidad, variedad, veracidad y valor.
3. Inventario y perfil inicial de las fuentes: grano, frecuencia, tipos, calidad, trazabilidad y riesgos.
4. Diagrama de arquitectura de alto nivel con fuentes, ingesta, Data Lake, procesamiento, serving, consumo y capacidades transversales.
5. Selección justificada del patrón batch, Lambda, Kappa o híbrido.
6. Mapeo de requisitos a componentes, incluyendo la relación entre las 5V y las decisiones de arquitectura.
7. Diseño del Data Lake: zonas, formatos, particiones, naming, retención, metadatos y reglas de promoción.
8. Flujo de datos batch y streaming desde las fuentes hasta las salidas, con herramientas específicas.
9. Flujo batch de referencia expresado con lógica MapReduce o equivalente conceptual.
10. Supuestos, riesgos iniciales, mitigaciones y decisiones todavía abiertas.
11. Estimación preliminar de esfuerzo, roles y recursos requeridos.
12. Repositorio inicial con README, convenciones y evidencia mínima de lectura, generación o exploración de datos.

5.3 Artefactos de entrega

Artefacto	Contenido mínimo
Documento de diseño	PDF o Markdown conciso con diagramas, tablas y decisiones; debe permitir una revisión rápida.
Repositorio	Estructura inicial, README, datos de muestra o instrucciones, evidencias y registro de decisiones.
Diagrama de arquitectura v1	Componentes, flujos, zonas, patrón elegido y capacidades transversales.
Matriz requisito-componente	Relación trazable entre objetivos/requisitos y los componentes propuestos.
Plan inicial	Supuestos, riesgos, mitigaciones, estimación de esfuerzo y próximos pasos.

5.4 Criterios de aceptación

- El problema, los usuarios y los criterios de éxito están formulados sin ambigüedad.
- La arquitectura responde a los requisitos y distingue claramente batch y streaming.
- Las zonas del Data Lake, formatos y particiones son coherentes con los datos provistos.
- El flujo MapReduce o equivalente muestra cómo se resolvería el procesamiento batch del caso.
- Los supuestos y riesgos son realistas y tienen mitigaciones propuestas.

- El repositorio y la documentación permiten continuar la implementación sin rehacer la fundación.

5.5 Rúbrica interna de la instancia

Dimensión	Indicadores	Peso
Comprensión y justificación	Problema, usuarios, preguntas, 5V y criterios de éxito	15 %
Arquitectura	Coherencia, responsabilidades, flujos y decisiones	25 %
Data Lake	Zonas, formatos, particiones, metadatos y retención	20 %
Datos	Inventario, perfil, calidad inicial y trazabilidad	15 %
Procesamiento batch	Lógica de transformación y correspondencia con MapReduce	10 %
Ingeniería y documentación	Repositorio, claridad, versionado, supuestos y riesgos	10 %
Defensa / respuesta al feedback	Capacidad de argumentar y priorizar correcciones	5 %

5.6 Producto posterior al feedback

Plan de correcciones

Con el feedback, cada equipo deberá versionar un plan de correcciones con prioridad, responsable, fecha objetivo y evidencia esperada. Este plan alimentará la arquitectura actualizada de la segunda entrega.

6 Segunda evaluación parcial

Implementación técnica y MVP end-to-end mínimo

FECHA Y HORA LÍMITE

Lunes 16 de noviembre de 2026 · 18:30 h

La jornada de clase del 28/09 se utilizará para revisión, feedback técnico y elaboración del plan de correcciones.

6.1 Objetivo

Demostrar que la propuesta dejó de ser únicamente conceptual y que existe un pipeline ejecutable, probado y documentado que conecta Landing -> Bronze -> Silver -> Gold -> Serving en Cassandra/AstraDB.

6.2 Alcance mínimo requerido

Componente	Evidencia requerida
Correcciones y arquitectura	Incorporar las correcciones priorizadas de la primera entrega y actualizar el diagrama para reflejar la implementación real.
Batch a Bronze	Ingestar al menos tres maestros, por ejemplo customers_orgs.csv, users.csv y billing_monthly.csv, a Parquet particionado con tipificación explícita, ingest_ts, source_file y deduplicación cuando aplique.
Streaming a Bronze	Procesar usage_events_stream/*.jsonl con esquema explícito, watermark, deduplicación por event_id, manejo de late data y checkpointing.
Silver	Limpiar y conformar eventos y al menos un maestro; realizar joins de enriquecimiento e implementar como mínimo tres features entre daily_cost_usd, requests, genai_tokens, carbon_kg u otras justificadas.
Calidad	Activar al menos tres reglas; incluir event_id no nulo/único, cost_usd_increment >= -0.01 con flag de anomalía y unit no nulo cuando exista value, o reglas equivalentes justificadas. Generar quarantine con muestras.
Gold	Construir el mart FinOps org_daily_usage_by_service con grano diario por organización y servicio, métricas y costos.
Serving	Crear un keyspace y una tabla query-first para el mart; cargar desde Spark y ejecutar dos consultas mínimas, incluyendo CQL y captura de resultados.
Analítica o ML	Incorporar un componente analítico o de Machine Learning cuando corresponda, o presentar una alternativa equivalente acordada y su plan de cierre.
Idempotencia	Demostrar re-ejecución sin duplicados mediante conteos antes/después, checkpoints, claves naturales o upserts.
Gobierno preliminar	Definir controles iniciales, metadatos, linaje, responsabilidades, acceso y evidencias.
Pruebas y reproducibilidad	Documentar puesta en marcha, configuración, logs, manejo de errores y pruebas.
Backlog final	Priorizar el trabajo restante para el MVP mediante categorías obligatorio, deseable y fuera de alcance.

6.3 Artefactos de entrega

- Repositorio con notebooks o código reproducible, estructura Bronze/Silver/Gold en Parquet, scripts CQL y README Quickstart con pasos exactos.

- Diagrama actualizado y consistente con la solución ejecutable.
- Log de decisiones sobre patrón Lambda/Kappa/híbrido, particiones, claves Cassandra y umbrales de anomalías.
- Evidencias de ejecución: logs, rutas, tamaños, conteos, muestras de quarantine y capturas de consultas.
- Backlog priorizado para completar el MVP final.

6.4 Criterios de aceptación

- Batch y streaming corren con los datos provistos.
- Las reglas de calidad y quarantine son efectivas y cuentan con ejemplos verificables.
- El mart FinOps existe en Gold y la tabla Cassandra/AstraDB está poblada.
- Se ejecutan dos consultas mínimas sobre AstraDB con resultados.
- La re-ejecución no duplica registros.
- El diagrama representa lo implementado y no componentes meramente aspiracionales.
- El README permite ejecutar la solución desde un entorno limpio.

6.5 Rúbrica interna de la instancia

Dimensión	Indicadores	Peso
Evolución del diseño	Correcciones y consistencia entre diagrama y producto	10 %
Ingesta	Reproducibilidad, validaciones y manejo de errores	15 %
Procesamiento Spark	Uso correcto de DataFrames, particiones, transformaciones y evidencia	25 %
Serving / persistencia	Modelo orientado al consumo y consultas	15 %
Analítica o ML	Utilidad, pipeline y métricas; o alternativa equivalente acordada	10 %
Calidad y gobierno	Controles iniciales, metadatos, linaje y acceso	10 %
Pruebas y reproducibilidad	Automatización, configuración, logs y README	10 %
Defensa / respuesta al feedback	Diagnóstico y plan de cierre	5 %

6.6 Producto posterior al feedback

Backlog final y congelamiento de alcance

Al finalizar la jornada del 16/11, el equipo deberá dejar un backlog final priorizado. Las funcionalidades opcionales no deben poner en riesgo la integración end-to-end, la reproducibilidad ni los requisitos mínimos de la instancia final.

7

Evaluación final

MVP integrado, demostración y defensa técnica

FECHA Y HORA LÍMITE

Lunes 7 de diciembre de 2026 · 21:30 h

La entrega debe contener todos los artefactos definitivos y una versión reproducible del MVP.

7.1 Objetivo

Entregar y defender un producto reproducible que ejecute el flujo end-to-end del caso Cloud Provider Analytics, desde las fuentes hasta el consumo o salida final. La defensa debe explicar arquitectura, decisiones, limitaciones, resultados y próximos pasos.

7.2 Alcance funcional y técnico obligatorio

1. Pipeline completo desde las fuentes necesarias hasta la capa de consumo, con ingesta batch y streaming operativa.
2. Arquitectura final consistente con la implementación real y con trazabilidad entre requisitos, datos, procesos y salidas.
3. Data Lake Landing/Bronze/Silver/Gold en Parquet, con particionamiento, promoción, calidad y evolución de esquema documentados.
4. Procesamiento Spark correcto, escalable para el alcance y evidenciado mediante logs, conteos y resultados.
5. Persistencia y serving en Cassandra/AstraDB con modelado query-first, tablas y consultas funcionales.
6. Analítica o Machine Learning integrado cuando el caso lo requiera, incluyendo métricas y limitaciones; de no aplicar, capacidad analítica equivalente acordada.
7. Controles de calidad, quarantine, metadatos, linaje, responsabilidades, seguridad y observabilidad.
8. Configuración externalizada, manejo de errores, idempotencia, pruebas y mecanismos de limpieza/reinicio.
9. Documentación técnica y presentación ejecutiva del resultado.
10. Demostración remota reproducible y defensa oral con participación de todos los integrantes cuando corresponda.

7.3 Capa Gold y dominios analíticos

Dominio	Mart de referencia	Grano	Contenido
FinOps	org_daily_usage_by_service	org_id, usage_date, service	Métricas de uso, requests y costos diarios.
FinOps	revenue_by_org_month	org_id, month	Revenue en USD, créditos, impuestos y FX aplicado.
FinOps	cost_anomaly_mart	org_id, date, service	Score o flag de anomalía y contexto del costo.
Soporte	tickets_by_org_date	org_id, date, severity	Conteos, SLA breach rate y CSAT promedio.
Producto / GenAI	genai_tokens_by_org_date	org_id, date	Tokens totales y costo estimado cuando existan.

Los nombres anteriores son referencias de diseño. La solución puede utilizar nombres equivalentes siempre que conserve el grano, permita responder las consultas obligatorias y documente la correspondencia.

7.4 Consultas mínimas que deben responderse desde Cassandra/AstraDB

1. Costos y requests diarios por organización y servicio en un rango de fechas.
2. Top-N servicios por costo acumulado en los últimos 14 días para una organización.
3. Evolución de tickets críticos y tasa de SLA breach por día durante los últimos 30 días.
4. Revenue mensual con créditos e impuestos aplicados, normalizado a USD.
5. Tokens GenAI y costo estimado por día, cuando existan.

Evidencia obligatoria

Para cada consulta se debe incluir el CQL, la justificación del modelo query-first y una captura o salida reproducible con resultados.

7.5 Artefactos finales

Artefacto	Contenido
Repositorio definitivo	Código, notebooks, jobs, configuración, pruebas, scripts CQL, datos de muestra/instrucciones y evidencias.
README Quickstart	Requisitos, versiones, instalación, configuración, ejecución completa, pruebas, limpieza y solución de problemas.
Documentación técnica	Arquitectura final, diccionario de datos, modelo de serving, calidad, gobierno, linaje, decisiones y limitaciones.
Presentación ejecutiva	Contexto de negocio, propuesta de ingeniería, arquitectura, tecnologías, proceso, resultados y consultas.
Video explicativo	Recorrido basado en la presentación y en la solución; debe mostrar el pipeline y los resultados del ETL/streaming/serving.
Defensa oral remota	Demostración y respuesta técnica sobre diseño, implementación, trade-offs, limitaciones y próximos pasos.

7.6 Criterios de aceptación final

- ☐ El flujo end-to-end se ejecuta y cumple el alcance mínimo del proyecto.
- ☐ Las cinco consultas se responden desde Cassandra/AstraDB con evidencias.
- ☐ La arquitectura y la documentación coinciden con el producto real.
- ☐ El pipeline es idempotente y puede reiniciarse o reprocesarse sin duplicar.
- ☐ Las reglas de calidad, quarantine, metadatos y linaje son verificables.
- ☐ El repositorio no contiene credenciales ni datos sensibles.
- ☐ La solución puede ser reproducida en forma remota mediante el Quickstart.
- ☐ La presentación, el video y la defensa comunican decisiones y resultados con un storytelling técnico coherente.

7.7 Rúbrica interna de la instancia

Dimensión	Indicadores	Peso
Funcionamiento end-to-end	Integración, estabilidad y cumplimiento del alcance mínimo	25 %
Coherencia arquitectónica	Trazabilidad entre problema, diseño e implementación	15 %
Procesamiento	Corrección, escalabilidad y uso pertinente de Spark	15 %
Persistencia y serving	Modelo de datos y respuesta a consultas	10 %
Analítica o ML	Valor aportado, métricas y limitaciones	10 %

Dimensión	Indicadores	Peso
Gobierno y calidad	Responsables, controles, linaje, seguridad y observabilidad	10 %
Ingeniería y reproducibilidad	Código, configuración, pruebas, documentación y automatización	10 %
Defensa	Claridad, síntesis y respuesta técnica	5 %

Nota sobre ponderaciones

Los porcentajes de las rúbricas corresponden al interior de cada instancia. Este documento no establece la ponderación global entre primera parcial, segunda parcial y evaluación final; dicha ponderación se registrará por el programa oficial y los criterios del docente.

8

Estándares de implementación y entrega

Reproducibilidad, documentación, calidad de ingeniería y modalidad remota

8.1 Estructura recomendada del repositorio

Carpeta / archivo	Propósito
README.md	Objetivo, arquitectura, requisitos, ejecución, pruebas y limitaciones.
docs/	Diagramas, decisiones, datos, gobierno y material de defensa.
data/	Datos de muestra o instrucciones para obtenerlos; nunca secretos.
src/ o jobs/	Código de ingesta, procesamiento, ML y serving.
notebooks/	Exploración y prácticas, separadas del código productivo cuando sea posible.
tests/	Pruebas de transformaciones, calidad y componentes.
config/	Configuración externalizada y ejemplos sin credenciales.
infra/	Docker, compose, scripts o manifiestos necesarios para ejecutar.
evidence/	Logs, capturas y resultados de las entregas.
DECISIONS.md	Decisiones, alternativas y justificaciones relevantes.

8.2 Reproducibilidad mínima

- Instrucciones completas desde un entorno limpio.
- Versiones de herramientas y dependencias.
- Configuración externalizada mediante archivos de ejemplo o variables de entorno.
- Ausencia de credenciales, tokens o datos sensibles en el repositorio.
- Comando o secuencia clara para ejecutar el pipeline.
- Datos de prueba suficientemente pequeños para una demostración remota.
- Evidencia de salida esperada y mecanismo de limpieza o reinicio.

8.3 Calidad del código y de los artefactos

- Nombres claros y consistentes.
- Separación entre configuración, lógica y datos.
- Funciones o transformaciones pequeñas y verificables.
- Manejo explícito de errores y registros de ejecución.
- Comentarios que expliquen decisiones, no que repitan el código.
- Diagramas actualizados y consistentes con el repositorio.
- Resultados regenerables, sin pasos manuales ocultos.

8.4 Estrategia para la demostración remota

- Mantener un dataset reducido y reproducible para la demo.
- Separar los problemas de instalación de la explicación conceptual mediante una guía previa.
- Conservar logs, capturas y salidas para diagnosticar fallos que no puedan reproducirse en vivo.
- Preparar un plan alternativo de demostración si fallan recursos locales, AstraDB o conectividad.
- La evidencia alternativa no reemplaza la necesidad de entregar una solución reproducible.

9

Anexos: checklists de entrega

Verificación operativa antes de cada fecha límite

9.1 Checklist de la primera evaluación · 28/09/2026 · 18:30 h

- ☐ Documento de diseño disponible en el canal de entrega.
- ☐ Repositorio accesible y versionado.
- ☐ Interpretación del caso y objetivos medibles.
- ☐ Análisis 5V.
- ☐ Inventario y perfil de fuentes.
- ☐ Arquitectura v1 y patrón justificado.
- ☐ Diseño Landing/Bronze/Silver/Gold.
- ☐ Flujos batch y streaming.
- ☐ Lógica MapReduce o equivalente.
- ☐ Matriz requisito-componente.
- ☐ Supuestos, riesgos, mitigaciones y estimación de esfuerzo.
- ☐ Evidencia mínima de lectura/exploración de datos.

9.2 Checklist de la segunda evaluación · 16/11/2026 · 18:30 h

- ☐ Correcciones de la primera entrega incorporadas.
- ☐ Arquitectura actualizada.
- ☐ Tres maestros batch en Bronze.
- ☐ Streaming con esquema, watermark, dedupe, late data y checkpoint.
- ☐ Silver para eventos y al menos un maestro.
- ☐ Tres features calculadas.
- ☐ Tres reglas de calidad y quarantine.
- ☐ Mart org_daily_usage_by_service en Gold.
- ☐ Keyspace y tabla Cassandra/AstraDB poblada.
- ☐ Dos consultas CQL con resultados.
- ☐ Idempotencia demostrada.
- ☐ README Quickstart, logs y evidencias.
- ☐ Backlog final priorizado.

9.3 Checklist de la evaluación final · 07/12/2026 · 21:30 h

- ☐ Pipeline completo batch y streaming.
- ☐ Data Lake y arquitectura final consistentes.
- ☐ Marts Gold para los dominios requeridos.
- ☐ Cinco consultas obligatorias desde Cassandra/AstraDB.
- ☐ Analítica o ML integrado, o alternativa acordada.
- ☐ Calidad, quarantine, metadatos, linaje, seguridad y observabilidad.
- ☐ Idempotencia, pruebas, configuración externa y manejo de errores.
- ☐ Repositorio final sin secretos.
- ☐ README Quickstart validado desde un entorno limpio.
- ☐ Documentación técnica y diccionario de datos.
- ☐ Presentación ejecutiva.
- ☐ Video explicativo.
- ☐ Demostración y defensa oral preparadas.

Cierre

El proyecto debe priorizar una solución coherente, ejecutable y defendible. Las extensiones opcionales solo aportan valor cuando no comprometen el funcionamiento end-to-end, la calidad, la reproducibilidad ni la claridad de las decisiones.